

Graphs

Introduction

A graph G is defined as an ordered set (V, E) , where $V(G)$ represents the set of vertices and $E(G)$ represents the edges that connect these vertices.

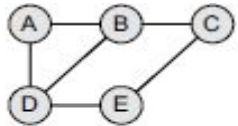


Figure 13.1 Undirected graph

$$V(G) = \{A, B, C, D \text{ and } E\}$$

$$E(G) = \{(A, B), (B, C), (A, D), (B, D), (D, E), (C, E)\}$$

Types : Directed Graph and Undirected Graph

In an **undirected graph**, edges do not have any direction associated with them.

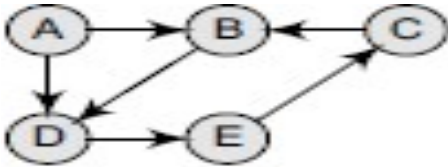


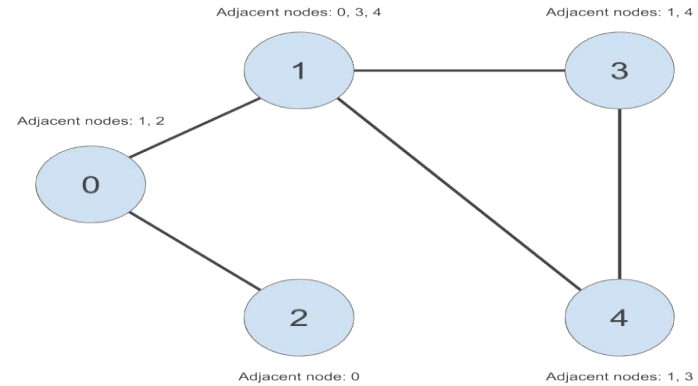
Figure 13.2 Directed graph

In a **directed graph**, edges form an ordered pair. If there is an edge from A to B , then there is a path from A to B but not from B to A . The edge (A, B) is said to initiate from node A (also known as initial node) and terminate at node B (terminal node).

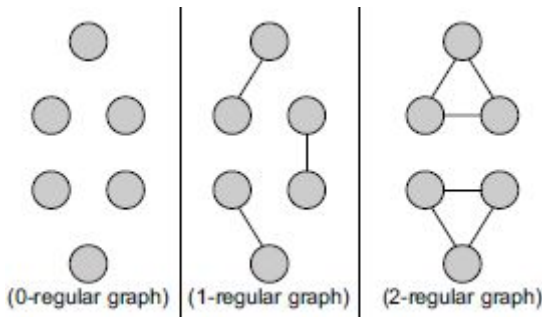
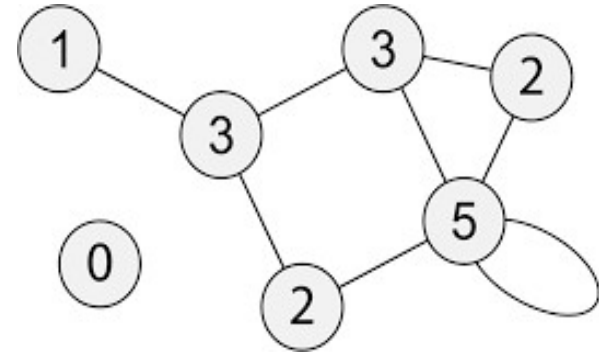
Graph Terminology

Adjacent nodes or neighbors

For every edge, $e = (u, v)$ that connects nodes u and v , the nodes u and v are the end-points and are said to be the adjacent nodes or neighbors



Degree of a node Degree of a node u , $\deg(u)$, is the total number of edges containing the node u . If $\deg(u) = 0$, it means that u does not belong to any edge and such a node is known as an **isolated node**.



Regular graph It is a graph where each vertex has the same number of neighbors. That is, every node has the same degree. A regular graph with vertices of degree k is called a k -regular graph or a regular graph of degree k .

Path A path P written as $P = \{v_0, v_1, v_2, \dots, v_n\}$, of length n from a node u to v is defined as a sequence of $(n+1)$ nodes. Here, $u = v_0$, $v = v_n$ and v_{i-1} is adjacent to v_i for $i = 1, 2, 3, \dots, n$.

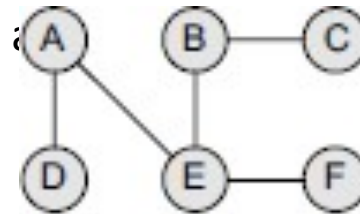
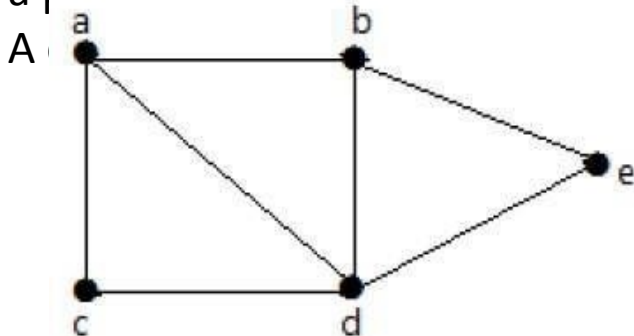
Closed path A path P is known as a closed path if the edge has the same end-points. That is, if $v_0 = v_n$.

Simple path A path P is known as a simple path if all the nodes in the path are distinct.

Cycle A closed simple path with length 3 or more. A cycle of length k is known as k -cycle.

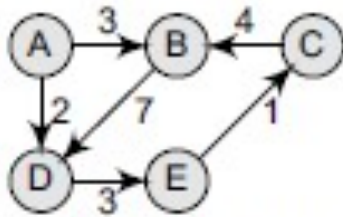
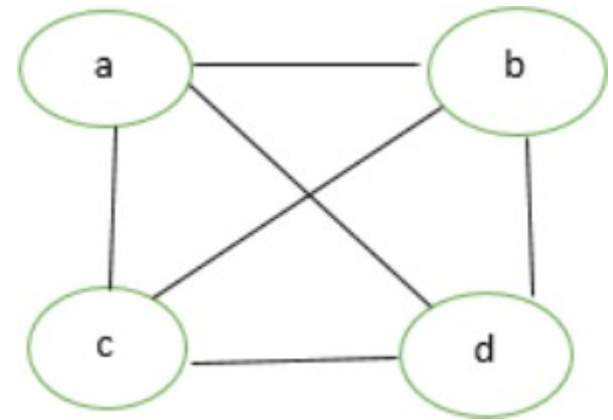
Connected graph A graph is said to be connected if for any two vertices (u, v) in V there is

a path from u to v . That is to say that there are **no isolated nodes in a connected graph**.
A graph that has any cycle is called a **tree graph**.



(b) Tree

Complete graph A graph is said to be complete , there is an edge between every single pair of the vertices in the graph. A complete graph is connected. A complete graph has $\frac{n(n-1)}{2}$ edges, where n is the number of nodes in G.



(c) Weighted graph

Labelled graph or weighted graph A graph is said to be labelled if every edge in the graph is assigned some data. In a weighted graph, the edges of the graph are assigned some non-negative weight or length.

Multiple edges Distinct edges which connect the same end-points are called multiple edges. That is, $e = (u, v)$ and $e' = (u, v)$ are known as multiple edges of G.

Loop An edge that has identical end-points is called a loop. That is, $e = (u, u)$. Loop constitute degree 2

Multi-graph A graph with multiple edges and/or loops is called a multi-graph.



(a) Multi-graph

Size of a graph The size of a graph is the total number of edges in it.

Directed Graphs

A directed graph G , also known as a *digraph*, is a graph in which every edge has a direction assigned to it. An edge of a directed graph is given as an ordered pair (u, v) of nodes in G . For an edge (u, v) ,

- The edge begins at u and terminates at v .
- u is known as the origin or initial point of e . Correspondingly, v is known as the destination or terminal point of e .
- u is the predecessor of v . Correspondingly, v is the successor of u .
- Nodes u and v are adjacent to each other.

Terminology of a Directed Graph

Out-degree of a node The out-degree of a node u , written as $\text{outdeg}(u)$, is the number of edges that originate at u .

In-degree of a node The in-degree of a node u , written as $\text{indeg}(u)$, is the number of edges that terminate at u .

Degree of a node The degree of a node, written as $\text{deg}(u)$, is equal to the sum of in-degree and out-degree of that node. Therefore, $\text{deg}(u) = \text{indeg}(u) + \text{outdeg}(u)$.

Isolated vertex A vertex with degree zero. Such a vertex is not an end-point of any edge.

Pendant vertex (also known as leaf vertex) A vertex with degree one.

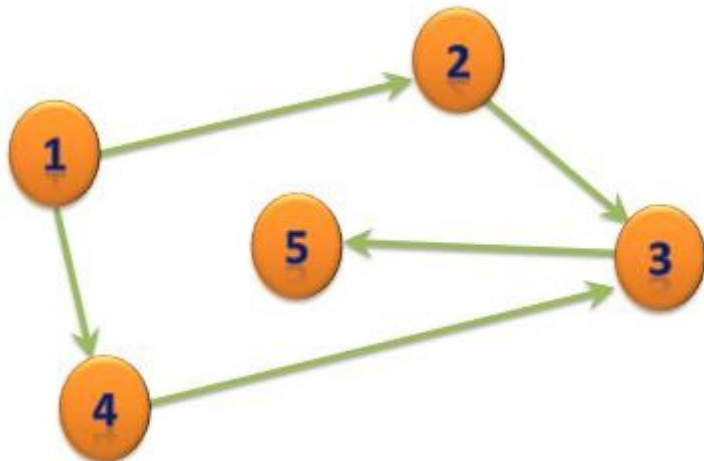
Source: A node u is called source if it has positive out degree but zero in degree.

Sink: A node u is called sink if it has positive in degree but zero out degree.

Reachability A node v is said to be reachable from node u , if and only if there exists a (directed) path from node u to node v .

Strongly connected directed graph A digraph is said to be strongly connected if and only if there exists a path between every pair of nodes in G

Unilaterally connected graph A digraph is said to be unilaterally connected if there exists a path between any pair of nodes u, v in G such that there is a path from u to v or a path from v to u , but not both.



Degree of 3 is 3

In Degree of 3 is 2

Out Degree of 3 is 1

Degree of 4 is 2

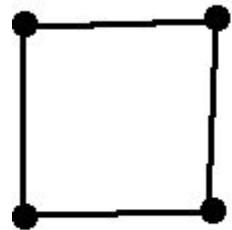
In Degree of 4 is 1

Weakly connected digraph A directed graph is said to be weakly connected if it is connected by ignoring the direction of edges.

Parallel/Multiple edges Distinct edges which connect the same end-points are called multiple edges.

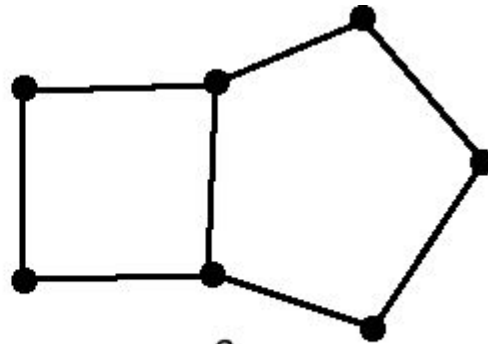
Simple directed graph A directed graph G is said to be a simple directed graph if and only if it has no parallel edges.

Subgraph: A subgraph G of a graph is graph G' whose vertex set and edge set subsets of the graph G



S

(Subgraph of G)



G

Graph Representation

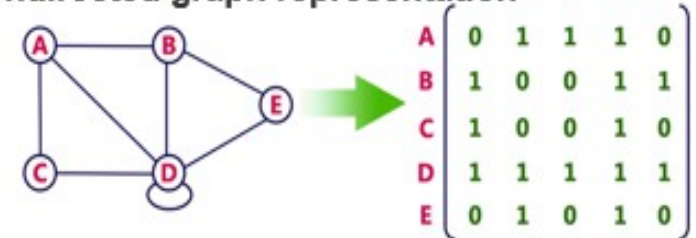
Adjacency Matrix

- Adjacency matrix is a sequential representation.
- It is used to represent which nodes are adjacent to each other. i.e. is there any edge connecting nodes to a graph.
- In this representation, we have to construct a $n \times n$ matrix A. If there is any edge from a vertex i to vertex j , then the corresponding element of A, $a_{ij} = 1$, otherwise $a_{ij} = 0$.
- If there is any weighted graph then instead of 1s and 0s, we can store the weight of the edge.

Directed graph representation

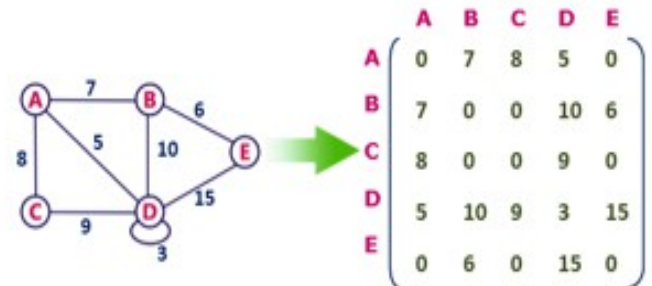


Undirected graph representation



Graph Representation

- Adjacency Matrix
- Incidence Matrix
- Adjacency List



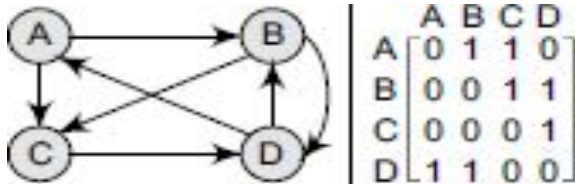


FIGURE 1

$$A^2 = A^1 \times A^1$$

$$A^2 = \begin{bmatrix} 0 & 1 & 1 & 0 \\ 0 & 0 & 1 & 1 \\ 0 & 0 & 0 & 1 \\ 1 & 1 & 0 & 0 \end{bmatrix} \times \begin{bmatrix} 0 & 1 & 1 & 0 \\ 0 & 0 & 1 & 1 \\ 0 & 0 & 0 & 1 \\ 1 & 1 & 0 & 0 \end{bmatrix} = \begin{bmatrix} 0 & 0 & 1 & 2 \\ 1 & 1 & 0 & 1 \\ 1 & 1 & 0 & 0 \\ 1 & 1 & 2 & 1 \end{bmatrix}$$

$$A^3 = \begin{bmatrix} 0 & 0 & 1 & 2 \\ 1 & 1 & 0 & 1 \\ 1 & 1 & 0 & 0 \\ 1 & 1 & 2 & 1 \end{bmatrix} \times \begin{bmatrix} 0 & 1 & 1 & 0 \\ 0 & 0 & 1 & 1 \\ 0 & 0 & 0 & 1 \\ 1 & 1 & 0 & 0 \end{bmatrix} = \begin{bmatrix} 2 & 2 & 0 & 1 \\ 1 & 2 & 2 & 1 \\ 0 & 1 & 2 & 1 \\ 1 & 2 & 2 & 3 \end{bmatrix}$$

$$A^4 = \begin{bmatrix} 2 & 2 & 0 & 1 \\ 1 & 2 & 2 & 1 \\ 0 & 1 & 2 & 1 \\ 1 & 2 & 2 & 3 \end{bmatrix} \times \begin{bmatrix} 0 & 1 & 1 & 0 \\ 0 & 0 & 1 & 1 \\ 0 & 0 & 0 & 1 \\ 1 & 1 & 0 & 0 \end{bmatrix} = \begin{bmatrix} 1 & 3 & 4 & 2 \\ 1 & 2 & 3 & 4 \\ 1 & 1 & 1 & 3 \\ 3 & 4 & 3 & 4 \end{bmatrix}$$

A^n (where n is the number of nodes in the graph) gives the number of paths of length n from node v_i to v_j .

$$B_r = A_1 + A_2 + A_3 + \dots + A_r$$

An entry in the i th row and j th column of matrix B_r gives the number of paths of length r or less than r from vertex v_i to v_j .

The main goal to define matrix B is to obtain the path matrix P . The path matrix P can be calculated from B by setting an entry $P_{ij} = 1$, if B_{ij} is non-zero and $P_{ij} = 0$

Transitive Closure of a Directed Graph

For a directed graph $G = (V, E)$, where V is the set of vertices and E is the set of edges, the transitive closure of G is a graph $G^* = (V, E^*)$. In G^* , for every vertex pair v, w in V there is an edge (v, w) in E^* if and only if there is a valid path from v to w in G .

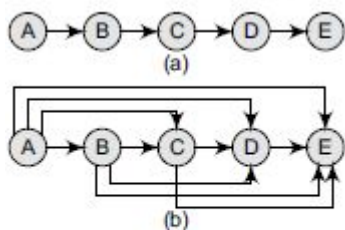


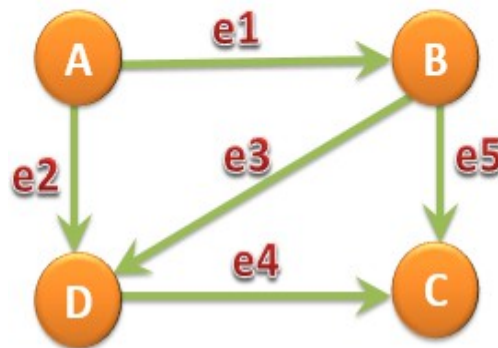
Figure 13.6 (a) A graph G and its
(b) transitive closure
 G^*

$$\begin{bmatrix} 0 & 1 & 1 & 0 \\ 0 & 0 & 1 & 1 \\ 0 & 0 & 0 & 1 \\ 1 & 1 & 0 & 0 \end{bmatrix} + \begin{bmatrix} 0 & 0 & 1 & 2 \\ 1 & 1 & 0 & 1 \\ 1 & 1 & 0 & 0 \\ 1 & 1 & 2 & 1 \end{bmatrix} + \begin{bmatrix} 2 & 2 & 0 & 1 \\ 1 & 2 & 2 & 1 \\ 0 & 1 & 2 & 1 \\ 1 & 2 & 2 & 3 \end{bmatrix} + \begin{bmatrix} 1 & 3 & 4 & 2 \\ 1 & 2 & 3 & 4 \\ 1 & 1 & 1 & 3 \\ 3 & 4 & 3 & 4 \end{bmatrix} = \begin{bmatrix} 3 & 6 & 6 & 5 \\ 3 & 5 & 6 & 7 \\ 2 & 3 & 3 & 5 \\ 6 & 8 & 7 & 8 \end{bmatrix}$$

Now the path matrix P can be given as:

$$P = \begin{bmatrix} 1 & 1 & 1 & 1 \\ 1 & 1 & 1 & 1 \\ 1 & 1 & 1 & 1 \\ 1 & 1 & 1 & 1 \end{bmatrix}$$

2. Incidence Matrix



Undirected graph

$$\begin{matrix} & \begin{matrix} e1 & e2 & e3 & e4 & e5 \end{matrix} \\ \begin{matrix} A \\ B \\ C \\ D \end{matrix} & \begin{bmatrix} 1 & 1 & 0 & 0 & 0 \\ -1 & 0 & 1 & 0 & 1 \\ 0 & 0 & 0 & -1 & -1 \\ 0 & -1 & -1 & 1 & 0 \end{bmatrix} \end{matrix}$$

$$B_{ij} = \begin{cases} 1 & \text{if vertex } v_i \text{ is incident with edge } e_j, \\ 0 & \text{otherwise.} \end{cases}$$

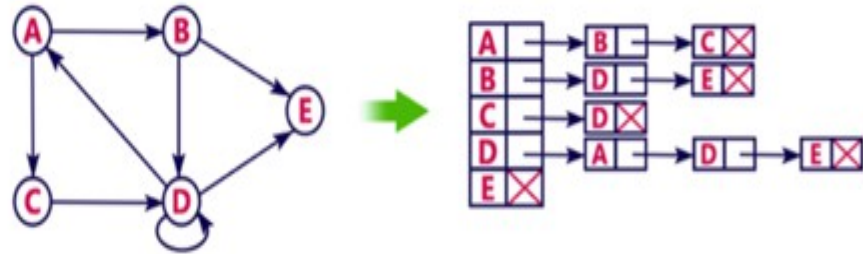
Adjacency List

- Adjacency list is a linked representation.

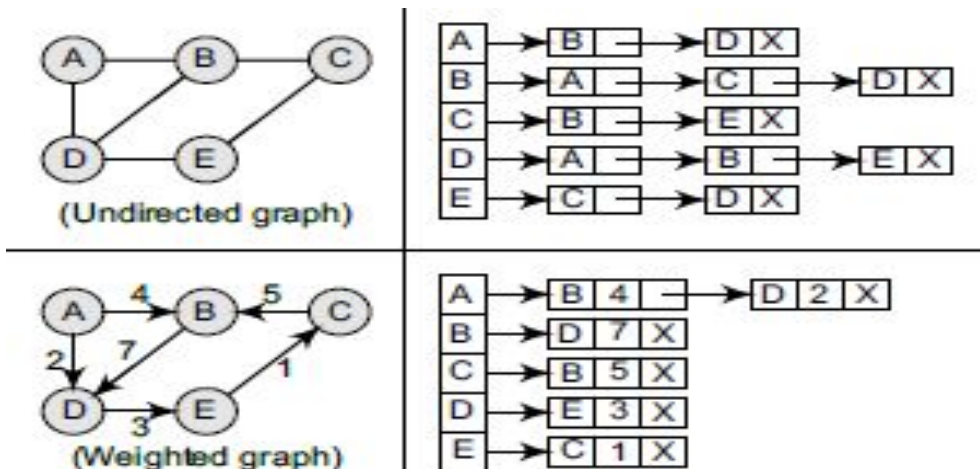
- In this representation, for each vertex in the graph, we maintain the list of its neighbors. It

means, every vertex of the graph contains list of

its adjacent vertices.



We can also implement this representation using array as follows:



Graph Traversal

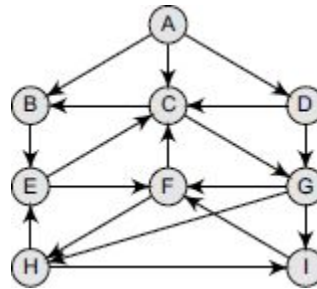
1. Breadth-first search
2. Depth-first search

While **breadth-first search** uses a **queue** as an auxiliary data structure to store nodes for further processing, the **depth-first search** scheme uses a **stack**. But both these algorithms make use of a variable **STATUS**.

Status	State of the node	Description
1	Ready	The initial state of the node N
2	Waiting	Node N is placed on the queue or stack and waiting to be processed
3	Processed	Node N has been completely processed

BFS

Step 1: SET STATUS = 1 (ready state) for each node in G
Step 2: Enqueue the starting node A and set its STATUS = 2 (waiting state)
Step 3: Repeat Steps 4 and 5 until QUEUE is empty
Step 4: Dequeue a node N. Process it and set its STATUS = 3 (processed state).
Step 5: Enqueue all the neighbours of N that are in the ready state (whose STATUS = 1) and set their STATUS = 2 (waiting state)
[END OF LOOP]
Step 6: EXIT



Adjacency lists	
A:	B, C, D
B:	E
C:	B, G
D:	C, G
E:	C, F
F:	C, H
G:	F, H, I
H:	E, I
I:	F

Find path P from A to I

(a) Add A to QUEUE and add NULL to ORIG.

FRONT = 0	QUEUE = A
REAR = 0	ORIG = \0

(b) Dequeue a node by setting FRONT = FRONT + 1 (remove the FRONT element of QUEUE) and enqueue the neighbours of A. Also, add A as the ORIG of its neighbours.

FRONT = 1	QUEUE = A	B	C	D
REAR = 3	ORIG = \0	A	A	A

(c) Dequeue a node by setting FRONT = FRONT + 1 and enqueue the neighbours of B. Also, add B as the ORIG of its neighbours.

FRONT = 2	QUEUE = A	B	C	D	E
REAR = 4	ORIG = \0	A	A	A	B

(d) Dequeue a node by setting FRONT = FRONT + 1 and enqueue the neighbours of C. Also, add C as the ORIG of its neighbours. Note that C has two neighbours B and G. Since B has already been added to the queue and it is not in the Ready state, we will not add B and only add G.

FRONT = 3	QUEUE = A	B	C	D	E	G
REAR = 5	ORIG = \0	A	A	A	B	C

(e) Dequeue a node by setting FRONT = FRONT + 1 and enqueue the neighbours of D. Also, add D as the ORIG of its neighbours. Note that D has two neighbours C and G. Since both of them have already been added to the queue and they are not in the Ready state, we will not add them again.

FRONT = 4	QUEUE = A	B	C	D	E	G
REAR = 5	ORIG = \0	A	A	A	B	C

(f) Dequeue a node by setting FRONT = FRONT + 1 and enqueue the neighbours of E. Also, add E as the ORIG of its neighbours. Note that E has two neighbours C and F. Since C has already been added to the queue and it is not in the Ready state, we will not add C and add only F.

FRONT = 5	QUEUE = A	B	C	D	E	G	F
REAR = 6	ORIG = \0	A	A	A	B	C	E

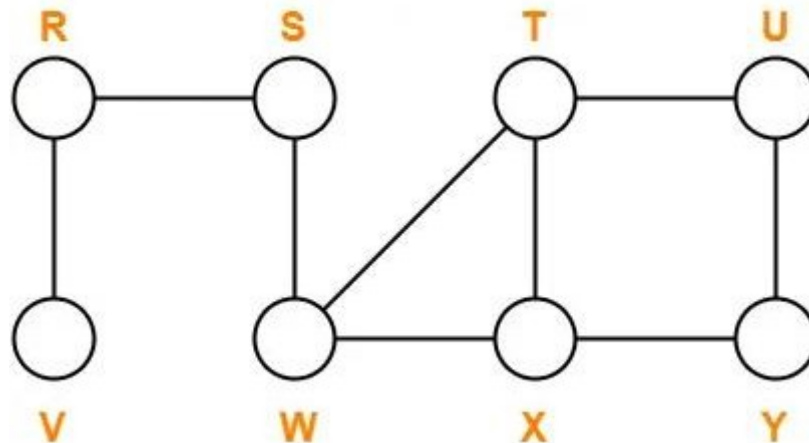
Figure 13.20 Algorithm for breadth-first search

Time complexity = $O(V+E)$

(g) Dequeue a node by setting $FRONT = FRONT + 1$ and enqueue the neighbours of G. Also, add G as the ORIG of its neighbours. Note that G has three neighbours F, H, and I.

FRONT = 6	QUEUE = A	B	C	D	E	G	F	H	I
REAR = 9	ORIG = \0	A	A	A	B	C	E	G	G

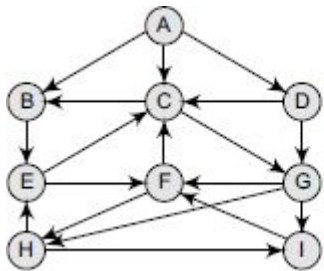
Traverse graph using
BFS



DFS

```

Step 1: SET STATUS = 1 (ready state) for each node in G
Step 2: Push the starting node A on the stack and set
        its STATUS = 2 (waiting state)
Step 3: Repeat Steps 4 and 5 until STACK is empty
Step 4: Pop the top node N. Process it and set its
        STATUS = 3 (processed state)
Step 5: Push on the stack all the neighbours of N that
        are in the ready state (whose STATUS = 1) and
        set their STATUS = 2 (waiting state)
        [END OF LOOP]
Step 6: EXIT
    
```



Adjacency lists	
A:	B, C, D
B:	E
C:	B, G
D:	C, G
E:	C, F
F:	C, H
G:	F, H, I
H:	E, I
I:	F

Consider the graph G given in Fig. 13.23. The adjacency list of G is also given. Suppose we want to print all the nodes that can be reached from the node H (including H itself).

(a) Push H onto the stack.

STACK: H

(b) Pop and print the top element of the STACK, that is, H. Push all the neighbours of H onto the stack that are in the ready state. The STACK now becomes

PRINT: H

STACK: E, I

(c) Pop and print the top element of the STACK, that is, I. Push all the neighbours of I onto the stack that are in the ready state. The STACK now becomes

PRINT: I

STACK: E, F

Time complexity = $O(V+E)$ (d) Pop and print the top element of the STACK, that is, F. Push all the neighbours of F onto the stack that are in the ready state. (Note F has two neighbours, C and H. But only C will be added, as H is not in the ready state.) The STACK now becomes

PRINT: F

STACK: E, C

(e) Pop and print the top element of the STACK, that is, C. Push all the neighbours of C onto the stack that are in the ready state. The STACK now becomes

PRINT: C

STACK: E, B, G

- (f) Pop and print the top element of the STACK, that is, G. Push all the neighbours of G onto the stack that are in the ready state. Since there are no neighbours of G that are in the ready state, no push operation is performed. The STACK now becomes

PRINT: G

STACK: E, B

- (g) Pop and print the top element of the STACK, that is, B. Push all the neighbours of B onto the stack that are in the ready state. Since there are no neighbours of B that are in the ready state, no push operation is performed. The STACK now becomes

PRINT: B

STACK: E

- (h) Pop and print the top element of the STACK, that is, E. Push all the neighbours of E onto the stack that are in the ready state. Since there are no neighbours of E that are in the ready state, no push operation is performed. The STACK now becomes empty.

PRINT: E

STACK:

Minimum spanning tree

A spanning tree of a connected, undirected graph G is a sub-graph of G which is a tree that connects all the vertices together. A graph G can have many different spanning trees

A minimum spanning tree or minimum cost spanning tree is that spanning tree, which covers all the vertices of the graph with minimum edges and the sum of the weight of those edges is minimum among other spanning trees of that graph.

The minimum spanning trees are mainly of two types:

- Prim's MST
- Kruskal's MST

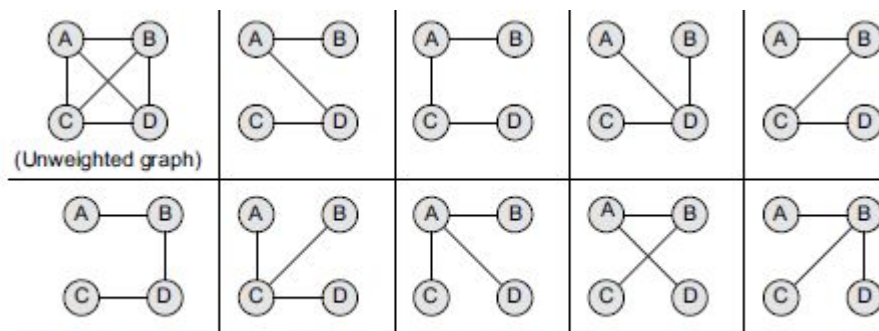


Figure 13.28 Unweighted graph and its spanning trees

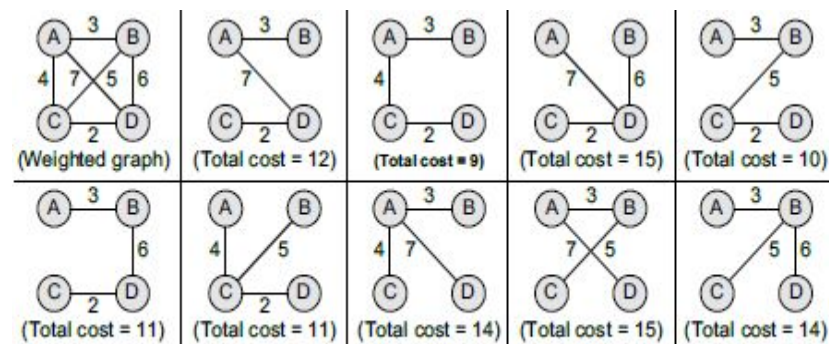


Figure 13.29 Weighted graph and its spanning trees

Prims MST algorithm

Prim's algorithm is a greedy algorithm that is used to form a minimum spanning tree for a connected weighted undirected graph. In other words, the algorithm builds a tree that includes

every vertex and a subset of the edges in such a way that the total weight of all the edges in the tree

is minimized. For this, the algorithm maintains three sets of vertices which can be given as below:

Tree vertices Vertices that are a part of the minimum spanning tree τ .

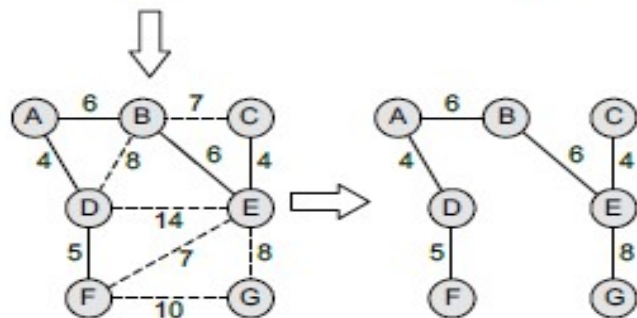
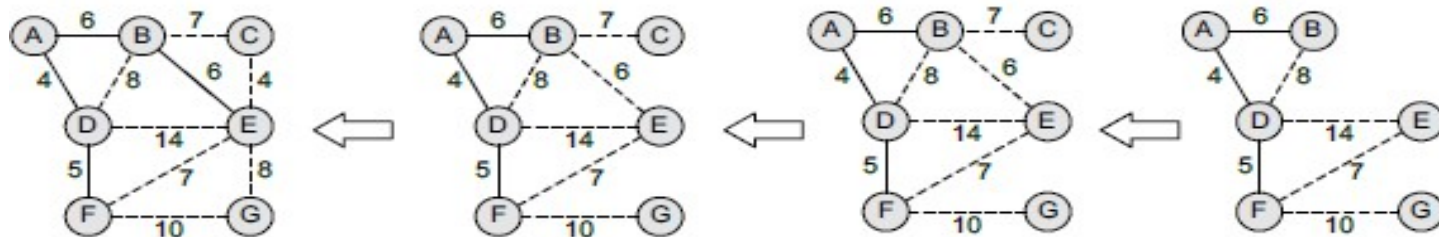
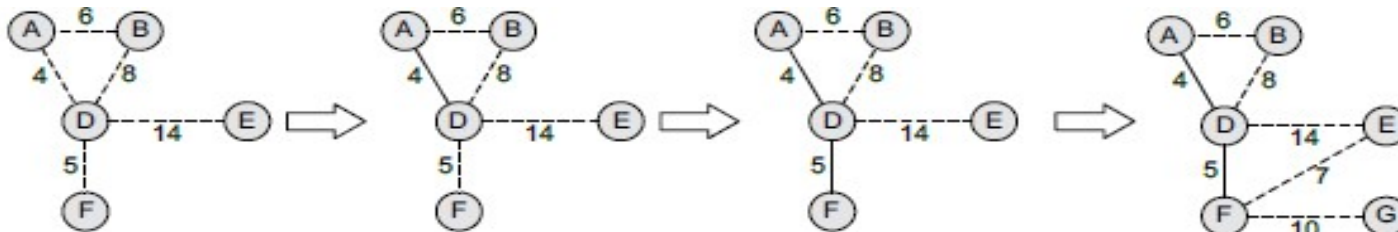
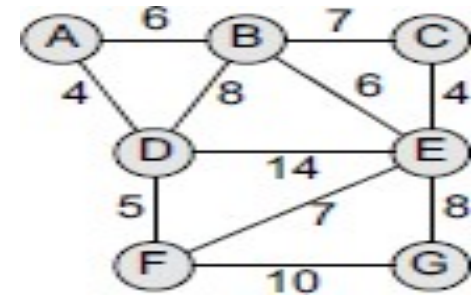
Fringe vertices Vertices that are currently not a part of τ , but are adjacent to some tree vertex.

Unseen vertices Vertices that are neither tree vertices nor fringe vertices fall under this category.

The steps involved in the Prim's algorithm are shown in Fig

```
Step 1: Select a starting vertex
Step 2: Repeat Steps 3 and 4 until there are fringe vertices
Step 3:   Select an edge e connecting the tree vertex and
          fringe vertex that has minimum weight
Step 4:   Add the selected edge and the vertex to the
          minimum spanning tree T
          [END OF LOOP]
Step 5: EXIT
```

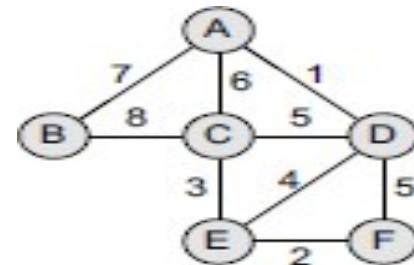
Construct a minimum spanning tree of the graph given in Fig. Start the Prim's algorithm from vertex D.



Kruskal's algorithm

Kruskal's algorithm is an example of a greedy algorithm. Kruskal's algorithm is used to find the minimum spanning tree for a connected weighted graph. The algorithm aims to find a subset of the edges that forms a tree that includes every vertex. The total weight of all the edges in the tree is minimized

```
Step 1: Create a forest in such a way that each graph is a separate
        tree.
Step 2: Create a priority queue Q that contains all the edges of the
        graph.
Step 3: Repeat Steps 4 and 5 while Q is NOT EMPTY
Step 4:     Remove an edge from Q
Step 5: IF the edge obtained in Step 4 connects two different trees,
        then Add it to the forest (for combining two trees into one
        tree).
        ELSE
            Discard the edge
Step 6: END
```

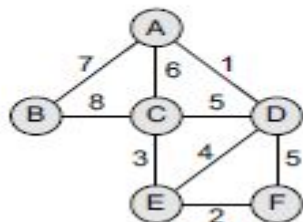


Initially, we have $F = \{\{A\}, \{B\}, \{C\}, \{D\}, \{E\}, \{F\}\}$

$MST = \{\}$

$Q = \{(A, D), (E, F), (C, E), (E, D), (C, D), (D, F), (A, C), (A, B), (B, C)\}$

Step 1: Remove the edge (A, D) from Q and make the following changes:

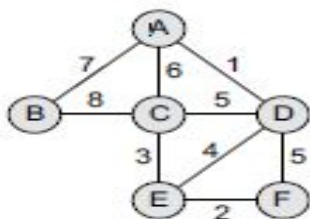


$F = \{\{A, D\}, \{B\}, \{C\}, \{E\}, \{F\}\}$

$MST = \{A, D\}$

$Q = \{(E, F), (C, E), (E, D), (C, D), (D, F), (A, C), (A, B), (B, C)\}$

Step 2: Remove the edge (E, F) from Q and make the following changes:

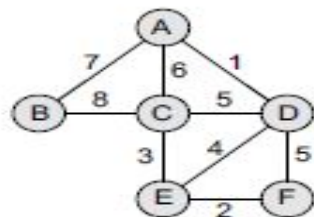


$F = \{\{A, D\}, \{B\}, \{C\}, \{E, F\}\}$

$MST = \{(A, D), (E, F)\}$

$Q = \{(C, E), (E, D), (C, D), (D, F), (A, C), (A, B), (B, C)\}$

Step 3: Remove the edge (C, E) from Q and make the following changes:

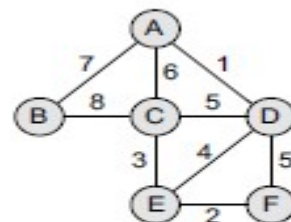


$F = \{\{A, D\}, \{B\}, \{C, E, F\}\}$

$MST = \{(A, D), (C, E), (E, F)\}$

$Q = \{(E, D), (C, D), (D, F), (A, C), (A, B), (B, C)\}$

Step 4: Remove the edge (E, D) from Q and make the following changes:



$$\begin{aligned} F &= \{\{A, C, D, E, F\}, \{B\}\} \\ \text{MST} &= \{(A, D), (C, E), (E, F), (E, D)\} \\ Q &= \{(C, D), (D, F), (A, C), (A, B), (B, C)\} \end{aligned}$$

Step 5: Remove the edge (C, D) from Q. Note that this edge does not connect different trees, so simply discard this edge. Only an edge connecting (A, D, C, E, F) to B will be added to the MST. Therefore,

$$\begin{aligned} F &= \{\{A, C, D, E, F\}, \{B\}\} \\ \text{MST} &= \{(A, D), (C, E), (E, F), (E, D)\} \\ Q &= \{(D, F), (A, C), (A, B), (B, C)\} \end{aligned}$$

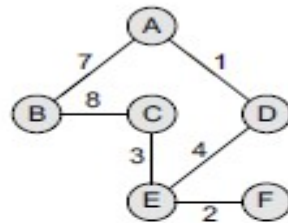
Step 6: Remove the edge (D, F) from Q. Note that this edge does not connect different trees, so simply discard this edge. Only an edge connecting (A, D, C, E, F) to B will be added to the MST.

$$\begin{aligned} F &= \{\{A, C, D, E, F\}, \{B\}\} \\ \text{MST} &= \{(A, D), (C, E), (E, F), (E, D)\} \\ Q &= \{(A, C), (A, B), (B, C)\} \end{aligned}$$

Step 7: Remove the edge (A, C) from Q. Note that this edge does not connect different trees, so simply discard this edge. Only an edge connecting (A, D, C, E, F) to B will be added to the MST.

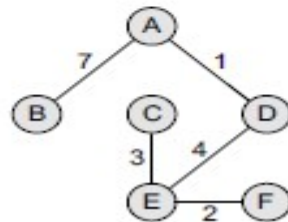
$$\begin{aligned} F &= \{\{A, C, D, E, F\}, \{B\}\} \\ \text{MST} &= \{(A, D), (C, E), (E, F), (E, D)\} \\ Q &= \{(A, B), (B, C)\} \end{aligned}$$

Step 8: Remove the edge (A, B) from Q and make the following changes:



$F = \{A, B, C, D, E, F\}$
 $MST = \{(A, D), (C, E), (E, F), (E, D), (A, B)\}$
 $Q = \{(B, C)\}$

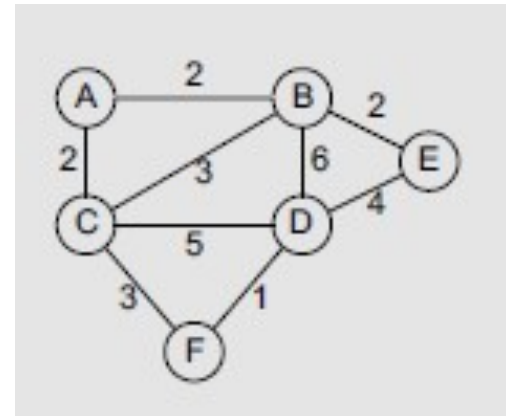
Step 9: The algorithm continues until Q is empty. Since the entire forest has become one tree, all the remaining edges will simply be discarded. The resultant MS can be given as shown below.



$F = \{A, B, C, D, E, F\}$
 $MST = \{(A, D), (C, E), (E, F), (E, D), (A, B)\}$
 $Q = \{\}$

The running time of Kruskal's and Prim's algorithm can be given as $O(E \log v)$, where E is the number of edges and v is the number of vertices in the graph.

Consider the graph given below. Find the minimum spanning tree of this graph using (a) Prim's algorithm, (b) Kruskal's algorithm,

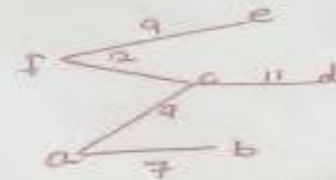


Dijkstra's Algorithm - Single Source Shortest Path - Greedy Algorithm

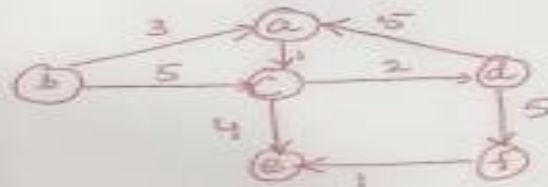


if $d(u) + c(u,v) < d(v)$
then $d(v) = d(u) + c(u,v)$

Source a	Destination				
	b	c	d	e	f
a	∞	∞	∞	∞	∞
a, b	7	∞	∞	∞	14
a, b, c	7	9	22	∞	14
a, b, c, d	7	9	20	∞	11
a, b, c, d, e	7	9	20	20	11
a, b, c, d, e, f	7	9	20	20	11



Question



Warshall's Algorithm

Graph: A directed graph with 4 nodes (1, 2, 3, 4). Edges: 1→2 (weight 8), 1→3 (weight 4), 2→3 (weight 1), 2→4 (weight 2), 3→4 (weight 9).

Matrix D^0 :

$$D^0 = \begin{bmatrix} 1 & 8 & 4 & \infty \\ \infty & 1 & \infty & 2 \\ \infty & \infty & 1 & 9 \\ \infty & \infty & \infty & 1 \end{bmatrix}$$

Matrix D^1 :

$$D^1 = \begin{bmatrix} 1 & 8 & 4 & \infty \\ \infty & 1 & \infty & 2 \\ \infty & \infty & 1 & 9 \\ \infty & \infty & \infty & 1 \end{bmatrix}$$

Matrix D^2 :

$$D^2 = \begin{bmatrix} 1 & 8 & 4 & \infty \\ \infty & 1 & \infty & 2 \\ \infty & \infty & 1 & 9 \\ \infty & \infty & \infty & 1 \end{bmatrix}$$

Matrix D^3 :

$$D^3 = \begin{bmatrix} 1 & 8 & 4 & \infty \\ \infty & 1 & \infty & 2 \\ \infty & \infty & 1 & 9 \\ \infty & \infty & \infty & 1 \end{bmatrix}$$

Matrix D^4 :

$$D^4 = \begin{bmatrix} 1 & 8 & 4 & \infty \\ \infty & 1 & \infty & 2 \\ \infty & \infty & 1 & 9 \\ \infty & \infty & \infty & 1 \end{bmatrix}$$

Path Matrices:

Path Matrix 1:

$$\begin{bmatrix} 1 & 2 & 3 & 4 \\ 1 & 1 & 2 & 1-2 \\ 2 & 2 & 2 & 2-3 \\ 3 & 3 & 3 & 3-3 \\ 4 & 4 & 4 & 4-4 \end{bmatrix}$$

Path Matrix 2:

$$\begin{bmatrix} 1 & 2 & 3 & 4 \\ 1 & 1 & 2 & 1-4 \\ 2 & 2 & 2 & 2-3 \\ 3 & 3 & 3 & 3-1-4 \\ 4 & 4 & 4 & 4-4 \end{bmatrix}$$

Path Matrix 3:

$$\begin{bmatrix} 1 & 2 & 3 & 4 \\ 1 & 1 & 2 & 1-4 \\ 2 & 2 & 2 & 2-3 \\ 3 & 3 & 3 & 3-1-4 \\ 4 & 4 & 4 & 4-4 \end{bmatrix}$$

Path Matrix 4:

$$\begin{bmatrix} 1 & 2 & 3 & 4 \\ 1 & 1 & 2 & 1-4 \\ 2 & 2 & 2 & 2-3 \\ 3 & 3 & 3 & 3-1-4 \\ 4 & 4 & 4 & 4-4 \end{bmatrix}$$



Vivekananda Institute
of Professional Studies



Vivekananda Institute
of Professional Studies

END